

Dans tous les exercices on suppose qu'une valeur faible correspond à une priorité plus élevée. On suppose qu'il n'y a que les processus indiqués dans le système et un seul processeur. Pour départager 2 processus, on fera l'ordonnancement par ordre alphabétique.

★ Exercice 1:

5 travaux A, B, C, D et E de durées respectives 10, 6, 2, 4 et 8 secondes, sont soumis à un ordinateur dans cet ordre, mais quasi simultanément. Ils ne font pas d'entrées-sorties.

Q 1: Compléter le tableau ci-dessous où DR est le délai de rotation, TA est le temps d'attente et TR est le temps de réponse. Pour le cas avec priorités, on donne $P(A) = 3$, $P(B) = 5$, $P(C) = 2$, $P(D) = 1$, $P(E) = 4$. Pour le Round-Robin, on suppose un quantum de 2 secondes (sans priorités).

Algorithmme	FIFO			SJF			Priorités			Round-Robin		
Schedule	A B C D E			C D B E A			D C A E B			A B C D E A B D E A B E A E A		
Métriques	DR	TR	TA	DR	TR	TA	DR	TR	TA	DR	TR	TA
A	10	0	0	30	20	20	16	6	6	30	0	20
B	16	10	10	12	6	6	30	24	24	22	2	16
C	18	16	16	2	0	0	6	4	4	6	4	4
D	22	18	18	6	2	2	4	0	0	16	6	12
E	30	22	22	20	12	12	24	16	16	28	8	20
Moyenne	19,2	13,2	13,2	14	8	8	16	10	10	20,4	4	14,4

A B "C" D E
A B "D" E
A "B" E
A "E"
"A"
Délai rot:
Temps où
on part *
durée
quantum

★ Exercice 2:

Q 2: Pour les processus du tableau ci-dessous, donnez le schéma d'ordonnancement par priorités dans les 2 cas : non préemptif et préemptif. La date d'arrivée et la durée sont exprimées en cycles.

Processus	Arrivée à	Durée	Priorité
A	0	4	6
B	1	3	5
C	2	3	3
D	3	5	4

★ Exercice 3: Sous Linux, soit un processus P qui fait des calculs pendant 60 ms crée 5 processus A, B, C, D et E pratiquement en même temps. Ensuite, il attend la fin de tous ses fils avant de faire 30 ms de calcul et terminer. Il met A, C et E dans la classe SCHED_RR avec les priorités 4, 2 et 2 respectivement. Il met B et D dans la classe SCHED_FIFO avec les priorités 3 et 1 respectivement.

Q 3: Donner le schéma d'ordonnancement correspondant sachant que :

- A : calcul(100)
- B : calcul(90), usleep(30), calcul(10)
- C : calcul(40), fait sched_setscheduler() pour passer en SCHED_NORMAL, calcul(30)
- D : calcul(70)
- E : calcul(30)

Le quantum de la classe SCHED_RR est 10. L'unité de temps utilisée est le milliseconde. Le schéma d'ordonnancement est à donner selon le format suivant : $X(T_X)-Y(T_Y)-\dots$ où X est le nom du processus et T_X est le temps alloué à ce processus.

★ Exercice 4: On considère une machine Linux (1 CPU). Soient 4 processus A, B, C et D créés en même temps avec les valeurs de nice 0, -4, -8 et 4 respectivement. Tous appartiennent à la classe SCHED_NORMAL (CFS) et ont le comportement suivant :

- A : $2 \times (1 \text{ ms calcul, } 8 \text{ ms E/S})$
- B : $2 \times (2 \text{ ms calcul, } 5 \text{ ms E/S})$
- C : 20 ms de calcul
- D : 10 ms de calcul

Q 4: Sur quoi CFS se base-t-il pour choisir le prochain processus à exécuter ? Sur le runtime, nice

Q 5: Donner le poids attribué à chaque processus et calculer la durée CPU (prorata des processus "runnable") initiale correspondante

$$L = 6\text{ms}$$

A : 1024

B 2501

C : 6100

D 423

Somme des poids : 10048

Slice init

$$A = 1024/10048 * 6 \leq 0.75$$

$$B = 1.49$$

$$C = 3.64$$

$$D = 423/10048 * 6 \leq 0.75$$

no.	Durée	CPU	Ready-Q	I/O-Queue	Justifications/Remarques
0	0		ABCD		
1	.75 (.25)	A	BCD		vruntime = .75
2	1.49	B	CD		vruntime = .61
3	3,64	C	D		vruntime = .61
4	.75	D			vruntime = 1.82
5		B	C A D	B	
6		C	A D		
7		A	D		
8		D			
9					
10					
11					
12					
13					
14					
15					
16					

CSF - Completely Fair Scheduler

Le CFS est l'ordonnanceur best-effort de Linux pour la classe `SCHED_NORMAL`. Il vise à réaliser un partage équitable du processeur, pondéré par des priorités, sans quantum fixe et sans prédiction du futur.

- Chaque processus reçoit une part du processeur proportionnelle à son poids. L'ordonnancement repose uniquement sur le temps CPU déjà consommé, corrigé par un facteur de priorité. Une valeur `nice` plus faible correspond à une priorité plus élevée.
- À chaque valeur `nice` est associé un poids w via la table `prio_to_weight` (listing 1).
- À un instant donné, pour un ensemble de processus *runnables*, la part de processeur reçue par un processus i est :

$$\text{part}_i = \frac{w_i}{\sum_{j \in \text{runnable}} w_j}.$$

- Le CFS introduit une période de référence L , appelée *scheduler latency*. Sur cette période, la slice cible d'un processus i est :

$$\text{slice}_i = L \times \frac{w_i}{\sum_{j \in \text{runnable}} w_j}.$$

Cette slice est recalculée à chaque décision d'ordonnancement.

- Pour éviter des passages processeur trop courts, une granularité minimale est imposée `min_granularity`
- Le `vruntime` est calculé comme suit :

$$\text{vruntime}_i = t_i \times \frac{1024}{w_i}.$$

où t_i est la durée CPU obtenue (cumulative).

Listing 1 – Correspondance nice-weight

```
const int sched_prio_to_weight[40] = {
/* -20 */      88761,    71755,    56483,    46273,    36291,
/* -15 */      29154,    23254,    18705,    14949,    11916,
/* -10 */      9548,     7620,     6100,     4904,     3906,
/* -5  */      3121,     2501,     1991,     1586,     1277,
/* 0   */      1024,      820,      655,      526,      423,
/* 5   */       335,      272,      215,      172,      137,
/* 10  */       110,       87,       70,       56,       45,
/* 15  */        36,       29,       23,       18,       15,
};
```